ROTA VITAL — PLANEJAMENTO DA REMODELAGEM ARQUITETURAL

Equipe / Squad: Gabriel Farias, Gabriel Nogueira, Lucas Mateus (@Lucas-Alves123), Mirella Emily, Myllena Cristiane
Instituição: CESAR School · Graduação em Análise e Desenvolvimento de Sistemas
Disciplina: Infraestrutura de Software (2026.2) · Trabalho U2 — Parte 2
Repositório Oficial: https://github.com/Lucas-Alves123/PI2

1. Introdução e Contexto do Projeto

O **Rota Vital** é uma plataforma centralizada de gestão logística de hemocomponentes. Ele automatiza o atendimento de solicitações hospitalares, a verificação de compatibilidade sanguínea ABO/Rh, o controle de estoque por validade (regra FEFO — *First Expired, First Out*), a expedição de guias e o rastreamento das condições de transporte em cadeia fria.

2. Remodelagem do Domínio com DDD (Seção A)

2.1. Classificação dos Subdomínios

- **Core (Diferencial Competitivo):**
- *Requisição e Alocação:* Gerencia solicitações hospitalares, valida compatibilidade ABO/Rh e executa alocação prioritária FEFO.
- *Estoque de Hemocomponentes:* Rastreabilidade individual de bolsas, controle de validades críticas, disponibilidade e descarte.
- *Expedição e Cadeia Fria:* Planejamento de rotas, geração de guias de transporte e garantia das condições térmicas.
- **Suporte (Necessários à Operação):**
- *Hospitais:* Cadastro de unidades de saúde parceiras e seus respectivos responsáveis técnicos.
- *Telemetria:* Ingestão contínua de leituras de temperatura e geolocalização dos veículos em trânsito.
- **Genérico (Soluções de Mercado):**
- *Identidade e Acesso:* Autenticação, perfis (Administrador, Atendimento, Estoque, Expedição) e autorização RBAC.
- *Notificação:* Mecanismos de disparo de alertas de não conformidade e avisos operacionais.

2.2. Bounded Contexts

1. **Contexto de Requisições:** Gestão de pedidos e ciclo de atendimento hospitalar.
2. **Contexto de Alocação:** Algoritmos de compatibilidade ABO/Rh e seleção FEFO.
3. **Contexto de Estoque:** Ciclo de vida físico das bolsas armazenadas em câmaras frias.
4. **Contexto de Expedição:** Agrupamento de bolsas em guias de remessa e despacho.
5. **Contexto de Hospitais:** Gestão de entidades demandantes e pontos de entrega.
6. **Contexto de Telemetria:** Processamento de séries temporais de sensores térmicos e GPS.
7. **Contexto de Identidade:** Governança de credenciais e segurança de acesso.

2.3. Glossário da Linguagem Ubíqua

- **Hospital:** Instituição de saúde habilitada a demandar hemocomponentes.
- **Unidade Solicitante:** Setor interno do hospital de onde parte o pedido (ex.: UTI, Bloco Cirúrgico).
- **Requisição:** Solicitação formal contendo componente, tipo sanguíneo, quantidade e urgência.
- **Alocação:** Ato de reservar e vincular formalmente uma bolsa a uma requisição ativa.
- **Compatibilidade:** Regra clínica sintética que determina se determinado doador pode suprir o receptor.
- **FEFO:** Priorização obrigatória da bolsa com menor data de validade remanescente.
- **Hemocomponente:** Fração terapêutica do sangue (Concentrado de Hemácias, Plasma, Plaquetas, Crioprecipitado).
- **Guia de Expedição:** Manifesto que acoberta o transporte físico entre o hemocentro e o hospital.
- **Cadeia Fria:** Faixa contínua de temperatura exigida para conservação dos hemocomponentes (2°C a 6°C).
- **Telemetria:** Medições automatizadas enviadas periodicamente pelo compartimento de transporte.
- **Alerta de Temperatura:** Notificação disparada quando os limites térmicos configurados são violados.
- **Termo com Duplo Sentido ("Bolsa"):**
- *No Estoque:* Unidade física rastreável com lote, tipo sanguíneo, data de coleta e prateleira.
- *Na Expedição:* Volume de carga unitizado associado a uma guia e monitorado pela telemetria.

2.4. Mapa de Contextos e Relacionamentos

- **Hospitais → Requisições:** Relação *Cliente-Fornecedor (C/F)* com *Camada Anticorrupção (ACL)*; a Requisição consome apenas identificadores (`hospitalId`), mantendo seu modelo livre de detalhes cadastrais.
- **Requisições ↔ Alocação:** Relação íntima de cooperação (*Shared Kernel / Parceria*) onde a Requisição delega a busca FEFO e recebe o conjunto de alocações.
- **Alocação → Estoque:** Relação *Upstream/Downstream*; o Estoque fornece a visão das bolsas disponíveis e impõe a invariante de reserva exclusiva.

- **Requisição → Expedição:** Relação orientada a eventos (*Publicador-Assinante*); a confirmação do pedido emite `RequisicaoAtendida`, consumida pela Expedição para montagem da guia.
- **Expedição → Telemetria:** *Cliente-Fornecedor*; a Telemetria associa fluxos contínuos de coordenadas e graus Celsius à guia de remessa ativa.
- **Telemetria / Estoque → Notificação:** *Pub/Sub*; eventos como `AlertaTemperatura` e `BolsaVencendo` disparam notificações.
- **Todos os Contextos → Identidade:** Relação *Conformista*; todas as camadas adotam a identidade propagada via Spring Security.

---

3. Modelo Tático de um Contexto Core (Seção B)

3.1. Agregado: Requisição (Contexto Core)

- **Raiz do Agregado:** `Requisicao` (entidade que assegura a integridade global do atendimento).
- **Entidades Internas:** `Alocacao` (representa o vínculo formal de uma bolsa específica com a requisição).
- **Objetos de Valor (Value Objects):**
- `TipoSanguineo` (imutável, composto por enum `TipoAbo` e enum `TipoRh`, encapsulando a regra `podeDoarPara()`).
- `Componente` (enum com Concentrado de Hemácias, Plasma Fresco Congelado, Plaquetas, Crioprecipitado).
- **Invariantes do Agregado:**
- *Compatibilidade Sanguínea:* Bolsas incompatíveis são terminantemente rejeitadas pelo agregado.
- *Prioridade FEFO:* Entre bolsas compatíveis, a alocação obriga o consumo da que possui menor tempo de validade.
- *Reserva Única:* Uma bolsa alocada não pode ser reutilizada por nenhuma outra requisição ativa (assegurado no agregado e por constraint única no banco).
- *Atendimento Completo:* O status só progride para `ATENDIDA` se a contagem de alocações ativas igualar a quantidade solicitada.

3.2. História de Usuário Âncora e Critérios de Aceite

> **Como** atendente do serviço de distribuição hospitalar,
> **Quero** solicitar bolsas de hemocomponentes indicando o tipo sanguíneo, componente e quantidade,
> **Para que** o sistema aloque automaticamente bolsas compatíveis com validade mais próxima, garantindo reserva exclusiva.

- **Critério 1 (Rejeição de Incompatíveis):** O sistema impede o vínculo de bolsas incompatíveis (ex.: bolsa AB+ para pedido O-).
- **Critério 2 (Ordenação FEFO):** Havendo múltiplas bolsas aptas, a de vencimento mais próximo é vinculada em primeiro lugar.
- **Critério 3 (Reserva Simultânea):** Tentativa de alocar bolsa já reservada lança violação de regra de negócio.

3.3. Cenário Executável em Gherkin

```
# language: pt
Funcionalidade: Alocação Inteligente de Bolsas por FEFO
  Contexto:
    Dado que existem três bolsas O+ de CH no estoque com vencimentos: 25/11, 15/11 e 01/12

  Cenário: Alocação prioritária da bolsa com vencimento mais próximo
    Quando uma requisição for registrada solicitando 1 bolsa O+ de CH
    Então a bolsa com validade em 15/11 deve ser alocada com sucesso
    E o status da requisição deve ser atualizado para "ATENDIDA"
```

---

4. Arquitetura em Três Camadas (Seção C)

4.1. Camadas Lógicas e Responsabilidades

1. **Apresentação:** Controllers web Spring MVC (`RequisicoesWebController`, `EstoqueWebController`, etc.) e templates Thymeleaf. Responsável por recepção de dados, validação de formato e renderização de respostas.
2. **Negócio:**
3. *Subcamada de Aplicação:* Serviços de caso de uso (`AlocacaoFefoService`, `AutenticacaoService`) que orquestram fluxos e transações (`@Transactional`).
4. *Subcamada de Domínio Puro:* Entidades (`Requisicao`, `Bolsa`, `GuiaExpedicao`), Value Objects e interfaces de repositório. O domínio é isolado de frameworks web e persistência.
5. **Dados:** Repositórios Spring Data JPA (`RequisicaoRepositoryJpa`, `BolsaRepositoryJpa`), migrações SQL versionadas (Flyway) e mapeamento ORM PostgreSQL.
6. **Regra de Dependência:** Apresentação depende da Aplicação; Aplicação coordena o Domínio e acessa contratos de Dados. O Domínio não conhece classes de camadas externas.
7. **Camada Lógica vs. Tier Físico:** As três camadas lógicas rodam integradas no mesmo artefato e processo JVM (Tier 1 — Aplicação Spring Boot), comunicando-se via rede TCP/IP com o servidor de banco de dados gerenciado (Tier 2 —

PostgreSQL 16).

4.2. Estrutura de Pacotes Proposta

```
br.cesar.rotavital
├── requisicao
│   ├── apresentacao (RequisicoesWebController)
│   ├── negocio
│   │   ├── aplicacao (AlocacaoFefoService)
│   │   └── dominio   (Requisicao, Alocacao, TipoSanguineo, RequisicaoRepositoryJpa)
│   └── dados
├── estoque
│   ├── apresentacao (EstoqueWebController)
│   ├── negocio/dominio (Bolsa, StatusBolsa, BolsaRepositoryJpa)
│   └── dados
├── expedicao (GuiaExpedicao, StatusExpedicao, GuiaExpedicaoRepositoryJpa)
├── telemetria (LeituraTelemetria, LeituraTelemetriaRepositoryJpa)
├── hospitais (Hospital, Unidade, HospitalRepositoryJpa)
└── identidade (Usuario, PerfilUsuario, SecurityConfig, AutenticacaoService)
```

5. Projeção para Microsserviços e ADRs (Seção D)

5.1. Contextos Candidatos a Microsserviços

- **Serviço de Telemetria:** Principal candidato a extração imediata devido à natureza volumétrica dos dados (leituras contínuas de sensores a cada minuto por veículo), necessitando de banco de séries temporais (ex.: TimescaleDB) e escala horizontal isolada.
- **Serviço de Notificações:** Candidato natural a operar como microserviço serverless acionado por eventos (AWS Lambda / SQS).
- **Core de Requisições e Estoque:** Mantido agrupado inicialmente para assegurar integridade transacional ACID nas reservas.

5.2. Padrão Saga Coreografada e Compensação

Em ambiente distribuído, o despacho hospitalar opera em saga: 1. *Serviço de Requisição* reserva as bolsas e emite `RequisicaoAtendida`. 2. *Serviço de Expedição* consome o evento e valida disponibilidade de veículo refrigerado. 3. **Fluxo de Compensação:** Caso não haja veículo disponível na frota, a Expedição publica `ExpedicaoFalhou`. O serviço de Estoque consome o evento e desfaz as reservas, restaurando as bolsas para o status `DISPONIVEL` e notificando o atendente.

5.3. Comparativo: Monólito Modular vs. Microsserviços

- **Unidade de Deploy:** Monólito utiliza 1 artefato único; Microsserviços demandam múltiplos pipelines e containers.
- **Consistência:** Monólito oferece transações ACID diretas; Microsserviços exigem consistência eventual (Saga/Outbox).
- **Custo e Operação:** Monólito apresenta baixa complexidade operacional e custo reduzido; Microsserviços exigem Service Mesh, tracing distribuído e maior orçamento.

5.4. Resumo das Decisões Arquiteturais (ADRs)

- **ADR 0001 (Decomposição):** Adoção de Monólito Modular na fase inicial do Rota Vital, preservando fronteiras de domínio estritas por pacotes para viabilizar futura separação de microsserviços sem atrito.
- **ADR 0002 (Stack Técnica):** Escolha de Java 21, Spring Boot 3.3.5, PostgreSQL 16 e Thymeleaf, garantindo alta produtividade, estabilidade e aderência às competências do squad.

6. Cenários de Carga e Orçamento de Nuvem (Seção E)

6.1. Dimensionamento via Lei de Little ($L = \lambda imes W$)

Assumindo tempo médio de processamento $W = 0,20 ext{ s}$ (200 ms): - **Cenário 1 — Baixo (Regional):** $\lambda = 2 ext{ req/s} ightarrow L = 2 imes 0,2 = 0,4$ requisições simultâneas.
Capacidade: 1 instância Spring Boot (512 MB) | Pool de conexões: 5. - **Cenário 2 — Moderado (Estadual):** $\lambda = 15 ext{ req/s} ightarrow L = 15 imes 0,2 = 3,0$ requisições simultâneas.
Capacidade: 2 instâncias Spring Boot (1 GB) + ALB | Pool de conexões: 20. - **Cenário 3 — Alto (Nacional):** $\lambda = 95 ext{ req/s} ightarrow L = 95 imes 0,2 = 19,0$ requisições simultâneas.
Capacidade: 4 instâncias Core + 2 instâncias Telemetria + ALB | Pool de conexões: 90.

6.2. Orçamento Detalhado (AWS On-Demand / Região us-east-1)

Consulta realizada em 10/11/2026 via AWS Pricing Calculator.

Cenário	Pedidos/mês	Recursos AWS	Custo Mensal (USD)	Custo / Pedido
1 — Baixo	1.500	1x ECS Fargate (0.25 vCPU) + RDS db.t3.micro (20 GB gp3)	**$24.57**	**$0.0164**
2 — Moderado	15.000	2x ECS Fargate (0.5 vCPU) + ALB + RDS db.t3.small (60 GB gp3)	**$92.18**	**$0.0061**

Cenário	Pedidos/mês	Recursos AWS	Custo Mensal (USD)	Custo / Pedido
3 — Alto	150.000	6x ECS Fargate + ALB + RDS db.t3.medium (Multi-AZ + Read Replica)	**$417.14**	**$0.0027**

Economia de Escala: O custo unitário por bolsa/pedido sofre redução de mais de **83%** (de $0.0164 para $0.0027), demonstrando a viabilidade econômica do modelo ao atingir escala nacional.

---

7. Conclusão

O planejamento da remodelagem do Rota Vital assegura o isolamento de regras críticas de triagem e FEFO, oferece transparência logística e define uma arquitetura limpa em três camadas, validada por testes automatizados e pronta para evolução distribuída.